# JetRacer simulation: quick start

Run the Isaac Sim JetRacer with the robot's SLAM and Nav2 stack, give it goals, drive it by hand and reset it. All commands run in `~/workspaces/ros2_ws`.

## 1. Start (three terminals)

**Terminal 1 simulation**

```
./run_isaac_sim.sh
```

Wait about a minute for `[jetracer_sim] running … in real time`.

The window opens on a camera in the room's upper corner, behind the car. Other views are in the viewport's camera menu (camera icon, top left): `Top` (a plan of the whole room, oriented like RViz) and `Corner_NE`, `Corner_NW`, `Corner_SW`, `Corner_SE`.

> **When the window asks "Save changes?" on close, always answer *Don't save*.** The furnished room, lights and camera wiring are added in memory on every start; saving writes them into the scene file and breaks the next start.

**Terminal 2 robot software**

```
./run_sim_slam.sh --nav --teleop
```

SLAM, the occupancy map and Nav2. Wait for these two lines:

1. `[grid] camera mount fitted to the floor …`
2. `Managed nodes are active`, which means Nav2 is ready.

Before that, `Timed out waiting for transform … odom` is normal. Don't drive yet.

**Terminal 3 keyboard + RViz**

```
ROS_DOMAIN_ID=31 ./run_teleop.sh sim
```

Opens RViz and drives the car from the keyboard. If you only want to watch: `ROS_DOMAIN_ID=31 ./run_rviz.sh sim` opens RViz alone.

Wait about 5 seconds after Nav2 is ready before sending the first goal.

## 2. Give a goal

1. In RViz, click **2D Goal Pose** in the top toolbar.
2. Click the spot on the map where the car should go. Keep the mouse button down, drag in the direction the car should face at the end, and release.

**Where you are on the map:** the map's origin is where the car was when terminal 2 started, which is the room centre, facing +x (the red axis). The room runs from −3 to +3 m in both directions, and the furniture stands along the walls.

### What you'll see

| | |
|---|---|
| **The plan** | A path appears at once. Space the car hasn't seen yet counts as free, so the path may cross it. |
| **Driving** | The car moves at about 0.15 m/s. It can reverse, and it turns in arcs, not on the spot. |
| **New obstacles** | When the camera sees something in the way (within 1 m), it appears on the map and the path changes to go around it. |
| **Finished** | Terminal 2 prints `Reached the goal!`. The car counts as arrived within 10 cm of the point; the final heading only has to be within about 35°. |

### Good first goals

- **1.2 m straight ahead, facing forward.** Easy.
- **0.8 m behind the start, facing backwards.** The car has to turn around.
- **About 1.6 m to the right (−y), facing −y.** The path goes around the grey bin.

## 3. More goals

| | |
|---|---|
| **Next goal after it arrives** | Click **2D Goal Pose** again and set the next goal. No reset is needed; the map and position carry on. |
| **Change goal while driving** | Just give a new goal. It replaces the current one straight away. |
| **Stop a goal without a new one** | Run this in any terminal; the car stops:<br>`source /opt/ros/humble/setup.bash`<br>`ROS_DOMAIN_ID=31 ros2 service call`<br>`/sim/navigate_to_pose/_action/cancel_goal \`<br>`  action_msgs/srv/CancelGoal "{}"`<br>If anything goes wrong, `Ctrl`+`C` in terminal 2 stops all driving at once. |

## 4. Drive by hand

Only drive by hand when no goal is active; otherwise Nav2 and the keyboard both send commands. Keeping terminal 3 open during goals is fine: it only sends commands while you press keys. Click on terminal 3 first so it gets the keys. Hold a key to keep moving; the car stops about 0.7 s after you let go.

| Key | Action |
|---|---|
| `↑` or `w` | forward |
| `↓` or `s` | backward |
| `←` `→` or `a` `d` | arc left / right in the last direction (forward at start and after space) |
| `q` / `e` | forward arc left / right |
| `z` / `c` | backward arc left / right |
| `space` | stop |
| `+` / `-` | faster / slower |

Holding `↑` and then pressing `←` keeps the car moving and curves it left (a terminal repeats only the last key held).

Letter keys need an English keyboard layout; arrows work in any layout.

**Recovering:** driving by hand is how you get out if the car is stuck or tracking is lost. In that case terminal 2 prints `[nav] no SLAM pose … holding the car still and cancelling the Nav2 goal`. Back away straight away (SLAM gives up on the current map after 3 s) until the camera view in RViz says `TRACKING` again, then give the goal again.

## 5. Reset

### Restart SLAM only (the car stays where it is)

1. `Ctrl`+`C` in terminal 2.
2. Run `./run_sim_slam.sh --nav --teleop` again.

The map starts empty, and the car's **current** position becomes the new origin. Goal coordinates now count from there.

### Full reset (the car goes back to the room centre)

1. `Ctrl`+`C` in terminal 2.
2. Close Isaac: `Ctrl`+`C` in terminal 1, or close the window and answer **Don't save**.
3. Start terminal 1, wait for `running`, then start terminal 2.

RViz (terminal 3) can stay open through both kinds of reset.

> If you're doing several runs in a row, check that nothing is left over before restarting:
>
> ```
> pgrep -fa "orb_slam3_node|planner_server"
> ```
>
> It should print nothing. Leftover processes fight over the same topics.

> **If a goal fails with** `GridBased: failed to create plan, exceeded maximum iterations`, the planner sees no way to the goal: it is inside an obstacle, or obstacles on the map close it off. Look at the pink cells in RViz around the car and the goal. (A ring of fake obstacles around the car caused this until 2026-10-10; it is fixed in `nav2_jetracer.yaml`.)

## 6. Options

| Command | What it does |
|---|---|
| `./run_isaac_sim.sh --headless` | no window (faster) |
| `./run_isaac_sim.sh --plain-room` | the original bare room (no furniture, ceiling or panel lights) |
| `./run_isaac_sim.sh --light 8000` | dimmer or brighter ceiling panels (default 12000) |
| `./run_sim_slam.sh --nav-wheels` | navigate on "wheel odometry", which in the simulation is the true pose (for comparison with SLAM) |
| `./run_sim_slam.sh` (no `--nav`) | SLAM and the map only, no Nav2 |

All terminals that talk to the simulation need `ROS_DOMAIN_ID=31`; the scripts in terminals 1 and 2 set it themselves. The full reference is in the ros2_ws `README.md` (sections 3 and 9).

---